

Unified ETL

Multi-channel sales data ingestion, standardization, and consolidation into a unified warehouse schema

1. EXTRACT

Multi-Format Ingestion

```
# Load diverse sources

in_store =
pd.read_csv('sales.csv',
encoding='latin-1')

online =
pd.read_json('online_sales.json')

partner =
pd.read_csv('partner_sales.csv')
```

2. TRANSFORM

Schema Standardization

```
# Standardize dates & channel

df['standard_date'] =
pd.to_datetime(...).dt.strftime('%Y-%m-%d')

df['channel'] = 'Online'

# Map columns to target schema

df = df.rename(columns={
    'order_id': 'transaction_id',
    'product_code': 'product_id',
    'units_sold': 'quantity'
})
```

3. LOAD

Warehouse Delivery

```
# Align columns & stack

final = pd.concat([
    in_store_final,
    online_final,
    partner_final
], ignore_index=True)
# Export PostgreSQL ready

final.to_csv('cleaned_warehouse_ready_data.csv',
index=False)
```

Dimensional and Fact table Creation

scripts to initialize and define the relationship constraints for the star schema tables

1. DIMENSIONS

Dimension Tables

```
-- Cleanup existing tables
DROP TABLE IF EXISTS Dim_Date CASCADE;
DROP TABLE IF EXISTS Dim_Product CASCADE;
DROP TABLE IF EXISTS Dim_Channel CASCADE;

CREATE TABLE Dim_Date (
    date_key INT PRIMARY KEY,
    full_date DATE,
    day INT,
    month INT,
    year INT
);

CREATE TABLE Dim_Product (
    product_key SERIAL PRIMARY KEY,
    product_id VARCHAR(50)
);

CREATE TABLE Dim_Channel (
    channel_key SERIAL PRIMARY KEY,
    channel_name VARCHAR(50)
);
```

2. FACTS

Fact Table

```
-- Cleanup existing fact table
DROP TABLE IF EXISTS Fact_Sales CASCADE;

CREATE TABLE Fact_Sales (
    sales_id SERIAL PRIMARY KEY,
    transaction_id VARCHAR(50),
    date_key INT REFERENCES Dim_Date(date_key),
    product_key INT REFERENCES Dim_Product(product_key),
    channel_key INT REFERENCES Dim_Channel(channel_key),
    quantity NUMERIC,
    unit_price NUMERIC,
    total_amount NUMERIC
);
```

Data Warehouse Schema Loading

Populating star schema dimension and fact tables from normalized staging datasets

1. DIMENSIONS

Loading Dimension Tables

```
-- Loading Dimension Tables from Staging

INSERT INTO Dim_Channel (channel_name)
SELECT DISTINCT channel FROM staging_sales
WHERE channel IS NOT NULL;

INSERT INTO Dim_Product (product_id)
SELECT DISTINCT product_id FROM staging_sales
WHERE product_id IS NOT NULL;

INSERT INTO Dim_Date (date_key, full_date, day, month, year)
SELECT DISTINCT TO_CHAR(standard_date, 'YYYYMMDD')::INT,
standard_date, EXTRACT(DAY FROM standard_date),
EXTRACT(MONTH FROM standard_date),
EXTRACT(YEAR FROM standard_date)
FROM staging_sales WHERE standard_date IS NOT NULL;
```

2. FACTS

Loading Fact Table

```
-- Loading Fact Table with Dimension Keys

INSERT INTO Fact_Sales (transaction_id, date_key,
product_key, channel_key, quantity, unit_price,
total_amount)

SELECT s.transaction_id,
TO_CHAR(s.standard_date, 'YYYYMMDD')::INT,
p.product_key, c.channel_key, s.quantity,
s.unit_price, s.total_amount

FROM staging_sales s
JOIN Dim_Product p ON s.product_id = p.product_id
JOIN Dim_Channel c ON s.channel = c.channel_name;
```

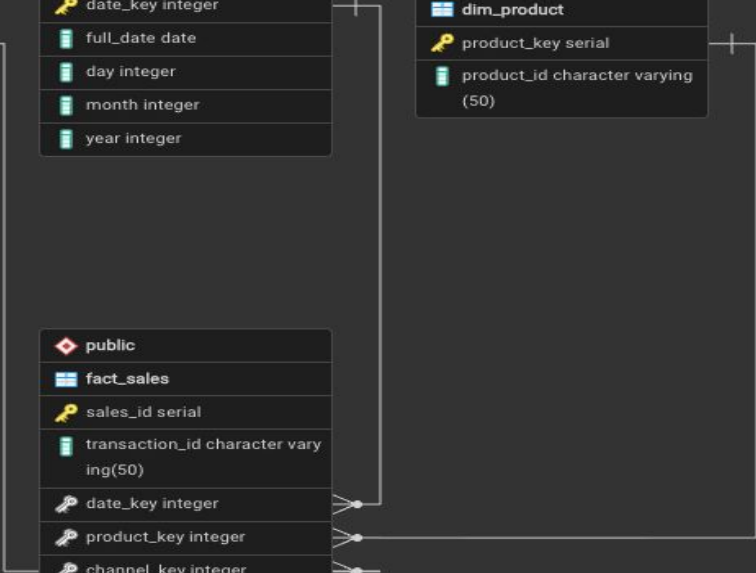
public
dim_channel
channel_key serial
channel_name character varying(50)

public
dim_date
date_key integer
full_date date
day integer
month integer
year integer

public
dim_product
product_key serial
product_id character varying(50)

public
fact_sales
sales_id serial
transaction_id character varying(50)
date_key integer
product_key integer
channel_key integer
quantity numeric
unit_price numeric
total_amount numeric

public
staging_sales
transaction_id character varying(50)
standard_date date
product_id character varying(50)
quantity numeric
unit_price numeric
total_amount numeric
channel character varying(50)



Warehouse Optimization and Partitioning

DDL scripts implementing lookup indexing, materialized aggregation views, and range-based table partitioning

1. INDEXING

Query Performance Tuning

```
-- 1. Indexing for faster lookups
CREATE INDEX idx_fact_date
ON Fact_Sales(date_key);

CREATE INDEX idx_fact_channel
ON Fact_Sales(channel_key);

CREATE INDEX idx_fact_product
ON Fact_Sales(product_key);
```

2. AGGREGATIONS

Materialized Views

```
-- 2. Materialized View for dashboard data
CREATE MATERIALIZED VIEW
mv_monthly_channel_sales AS
SELECT d.year, d.month, c.channel_name,
SUM(f.total_amount) AS total_revenue
FROM Fact_Sales f
JOIN Dim_Date d ON f.date_key = d.date_key
JOIN Dim_Channel c ON f.channel_key =
c.channel_key
GROUP BY d.year, d.month, c.channel_name;
```

3. PARTITIONING

Range Partitioning

```
-- 3. Partitioning Fact Table by Year
CREATE TABLE Fact_Sales_Partitioned (
sales_id SERIAL, transaction_id VARCHAR(50),
date_key INT, product_key INT,
channel_key INT, quantity NUMERIC,
unit_price NUMERIC, total_amount NUMERIC
) PARTITION BY RANGE (date_key);

CREATE TABLE sales_2023 PARTITION OF
Fact_Sales_Partitioned FOR VALUES FROM
(20230101) TO (20240101);

CREATE TABLE sales_2024 PARTITION OF
Fact_Sales_Partitioned FOR VALUES FROM
(20240101) TO (20250101);
```

Analytical OLAP Operations on Star Schema

DDL and Query scripts illustrating Roll-up, Drill-down, Slice, and Dice operations for multi-dimensional warehouse analysis

1. ROLL-UP

Yearly Aggregation

```
-- Summarize Month to Year
SELECT d.year, SUM(f.total_amount)
AS yearly_revenue
FROM Fact_Sales f
JOIN Dim_Date d ON f.date_key =
d.date_key
GROUP BY ROLLUP (d.year)
ORDER BY d.year;
```

2. DRILL-DOWN

Monthly Breakdown

```
-- Break Year into Months
SELECT d.year, d.month,
SUM(f.total_amount) AS
monthly_revenue
FROM Fact_Sales f
JOIN Dim_Date d ON f.date_key =
d.date_key
WHERE d.year = 2024
GROUP BY d.year, d.month
ORDER BY d.month;
```

3. SLICE

Online Sales Filter

```
-- Filter by one dimension
SELECT p.product_id,
SUM(f.total_amount) AS total_revenue
FROM Fact_Sales f
JOIN Dim_Product p ON f.product_key
= p.product_key
JOIN Dim_Channel c ON f.channel_key
= c.channel_key
WHERE c.channel_name = 'Online'
GROUP BY p.product_id
ORDER BY total_revenue DESC;
```

4. DICE

Online & 2024 Filter

```
-- Filter by two dimensions
SELECT p.product_id,
SUM(f.total_amount) AS total_revenue
FROM Fact_Sales f
JOIN Dim_Product p ON f.product_key =
p.product_key
JOIN Dim_Channel c ON f.channel_key =
c.channel_key
JOIN Dim_Date d ON f.date_key =
d.date_key
WHERE c.channel_name = 'Online'
AND d.year = 2024
GROUP BY p.product_id
ORDER BY total_revenue DESC;
```

Query Performance & Optimization Testing

Evaluating execution plans and latency differences between on-the-fly table joins and materialized views

1. BEFORE OPTIMIZATION

Raw Fact Table Query

```
-- Forces database to scan and join on the fly

EXPLAIN ANALYZE
SELECT d.year, d.month, c.channel_name,
       SUM(f.total_amount) as total_revenue
FROM Fact_Sales f
JOIN Dim_Date d ON f.date_key = d.date_key
JOIN Dim_Channel c ON f.channel_key = c.channel_key
GROUP BY d.year, d.month, c.channel_name;
```

2. AFTER OPTIMIZATION

Materialized View Query

```
-- Directly queries pre-aggregated warehouse cache

EXPLAIN ANALYZE
SELECT * FROM mv_monthly_channel_sales;
```